

Data Documentation

1. Chicago Data Portal

a. [Transportation Network Providers - Trips \(2025-\)](#)

This dataset serves as the foundation of our pipeline, containing all rideshare trips within the City of Chicago for 2025. We ingest this raw data at the start of our process, subsequently filtering and grouping it to prepare the necessary inputs for our API calls.

b. [Boundaries - Neighborhoods](#)

This dataset provides the polygon boundaries for Chicago's neighborhoods. By spatially joining these boundaries with our trip coordinate data, we mapped each trip's origin and destination to a specific neighborhood name. This dataset was also instrumental in rendering the geographic layers of our interactive map.

2. Google Routes API

We integrated the Google Routes API to generate hypothetical public transit alternatives for the rideshare trips in our dataset. This allowed us to retrieve comparative metrics, including total travel time, specific transit modalities (bus, rail, etc.), and distance.

Project Structure

Our repository is organized into specialized directories to handle everything from raw data ingestion to final visualization:

- `data_processing/`: The engine of the project. It houses scripts that transform raw Chicago Data Portal downloads into API-ready CSVs. It also contains the logic for joining processed rideshare data with transit data retrieved from our API.
- `api/`: Contains the core program for executing cached API requests. It parses responses and outputs CSVs specifically formatted for the `data_processing` join scripts.
- `data/`: The central storage hub for all CSV files used and generated throughout the pipeline.
- `dashboard/`: Contains the code for our Dash application. This includes a dedicated sub-folder for our Altair-based visualizations.
- `milestones/`: A record of our project's evolution, containing development notes and key documentation (including this file).
- `tests/`: Contains our pytest suite to ensure functional integrity across the various modules of the program.

To run the app, the `data_processing` and `api` folders can safely be ignored - the processed data exists in a zipped file in the `data` folder, and the app can be run based on that data. However, instructions to run the data processing code in the command line can be found within the `data_processing` folder.

Team Responsibilities

Sabrina spearheaded the initial data exploration and filtering process. Later, she transitioned the project's cleaning logic from experimental Jupyter notebooks into the structured architecture of `make_csv.py`. She was instrumental in developing the core data pipeline, co-authoring the rideshare-to-transit merge function and writing the `data_processing/__main__.py` script that runs the program. On the front-end, Sabrina contributed essential helper functions for data

transformation and led the exploratory visualization phase, focusing on data distribution, seasonality, and she built and iterated upon the entire Dash app!

Molly worked on the technical strategy for external integrations, evaluating the Google Routes API and navigating its pricing structure to ensure strategic, cost-effective usage. They collaborated closely with Sabrina on the initial cleaning and filtering scripts, wrote the bulk of the data processing code, and developed the rationale and code for grouping data for API calls. In addition to co-developing the `join.py` merge function, Molly was responsible for ongoing troubleshooting and general bug fixes across the repository. To wrap up the project, they provided the final synthesis of the team's findings by writing text for the dash app and documentation.

Sarah served as the architect of the project's API infrastructure, writing the robust code necessary to interface with Google Routes and manage cached responses. She also developed the interactive map, which is a major highlight of the dashboard, allowing users to explore our dataset in a really intuitive way. Beyond these core contributions, Sarah stepped in to assist with complex visualizations and providing critical code support across the repository whenever specialized troubleshooting or additional features were required, including reorganizing scripts and adding neighborhood data using a geospatial merge.

Waleed contributed to the project's data acquisition and visual analytics. He assisted with the final selection of the Google Routes API, which the team utilized to gather route-level comparisons between rideshare and public transit. Waleed's primary focus was the development of several Altair-based visualizations within the dashboard. His work included bar charts of high-volume pickup neighborhoods, scatter plots examining the relationship between trip distance and demand, and a heatmap designed to highlight connectivity patterns and transit penalties across major neighborhood corridors.

Final Thoughts

Our analysis reveals that rideshare trips are almost universally faster than their public transit counterparts. While rideshare is significantly more expensive, with nearly all neighborhood corridors exceeding the \$2.50 CTA fare and high-cost routes surpassing \$70, the average user appears to be making a rational choice regarding their time, if not their money. Ultimately, the time savings threshold for choosing rideshare over transit likely scales inversely with personal income.

A significant portion of the most expensive and longest trips are concentrated around O'Hare International Airport and, to a lesser extent, Midway. These hubs remain outliers in terms of demand, cost, and distance. In addition to the airports, the Loop and its immediate neighbors (such as River North and the West Loop) are the most frequent pickup locations.

Interestingly, many of the most popular corridors maintain a relatively low "transit penalty." This indicates that many Chicagoans opt for rideshare even when the transit alternative is not significantly longer (less than twice as long). While our analysis identifies several target

corridors for service improvements, it also suggests that efficiency alone may not shift behavior for everyone. However, localized service improvements remain vital; as research suggests, high-quality service in one part of the network can create a "halo effect" that improves public perception and ridership for the entire system.

In exploring our interactive map, we observed that many rides with the highest transit penalties occur between adjacent neighborhoods. This suggests that the CTA's current spoke-and-hub model may not sufficiently serve shorter, lateral trips. Furthermore, a large percentage of rideshare trips are under one mile, highlighting the "last-mile problem." These micro-trips might be more effectively replaced by expanded pedestrian and bike infrastructure rather than new fixed transit routes.

Finally, we observed that neighborhoods with the poorest transit alternatives for their top five destinations are often those already underserved in other capacities. These neighborhoods are predominantly located on the South and West Sides, including Chatham, Englewood, and Little Village.

To identify specific candidates for service improvement, we invite you to explore our interactive map. Look for popular routes with poor transit alternatives, highlighted in orange and red. How might the City of Chicago better connect these communities?